

PART 0 — Pehle Basics Clear Karte Hain

Chess Engine Actually Kya Hota Hai?

Socho tum ek **grandmaster** ho. Jab tumhari turn aati hai, tum kya karte ho?

1. **Saare possible moves** sochte ho (maybe 30-40 moves)
2. Har move ke baad **opponent ke possible replies** sochte ho (30-40 each)
3. Aur aage bhi... (this creates a **tree** 🌳)
4. Har branch ko **score** dete ho ("agar main ye chalaun toh mera position acha hai ya bura")
5. Sabse acha score wala move chunte ho

Bas! Ye hi chess engine hai. Computer bhi yahi karta hai — bas bahut fast, aur bahut deep.

Analogy: Ye aise hi hai jaise tum ek shatranj ka **family tree** bana rahe ho — har node ek position, har branch ek move.

FEN Kya Hai?

FEN = Forsyth-Edwards Notation. Ye ek **single line string** hai jo pura chess board describe karti hai.

```
text

rnbqkbnr/pppppppp/8/8/4P3/8/PPPP1PPP/RNBQKBNR b KQkq e3 0 1
```

Isko aise padho (5 parts):

Part	Example	Matlab
1	<code>rnbqkbnr/pppppppp/...</code>	8 rows ka board (lowercase = black, UPPERCASE = white)
2	<code>b</code>	Kiski turn hai (<code>b</code> = black, <code>w</code> = white)
3	<code>KQkq</code>	Castling rights (King-side, Queen-side for both)
4	<code>e3</code>	En-pasant target square
5	<code>0 1</code>	Halfmove clock & fullmove number

Analogy: FEN = chess position ka **Aadhaar card**. Har position ki unique identity.

Important: Engine ke liye sirf pehle **4 parts** matter karte hain. Last 2 (clocks) ignore karta hai — kyunki same position ho sakti hai clock alag hone pe. Isliye `getPositionKey()` sirf pehle 4 tokens use karta hai.

Web Worker Kya Hai? (Ye Sabse Important Concept Hai)

Problem: Agar tum heavy calculation **main thread** (jahan UI chalti hai) pe karoge, toh **screen freeze ho jayegi**. Button click nahi hoga, animation ruk jayega — user ko lagega app hang ho gaya.

Solution: Browser mein **2 threads** hote hain:

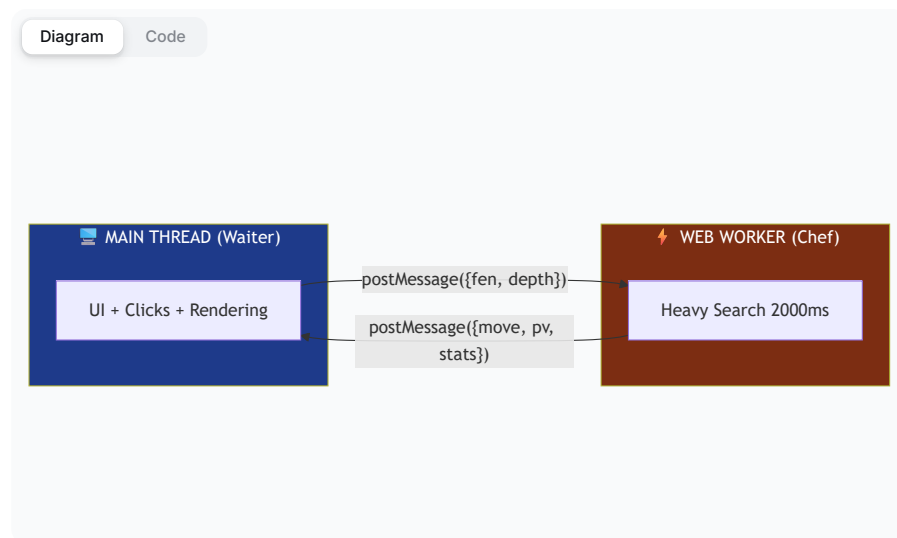
- **Main Thread:** UI, clicks, rendering
- **Web Worker Thread:** Background heavy math

Dono **messages** bhej ke baat karte hain (`postMessage`).

Analogy: Tum ek **restaurant** chala rahe ho.

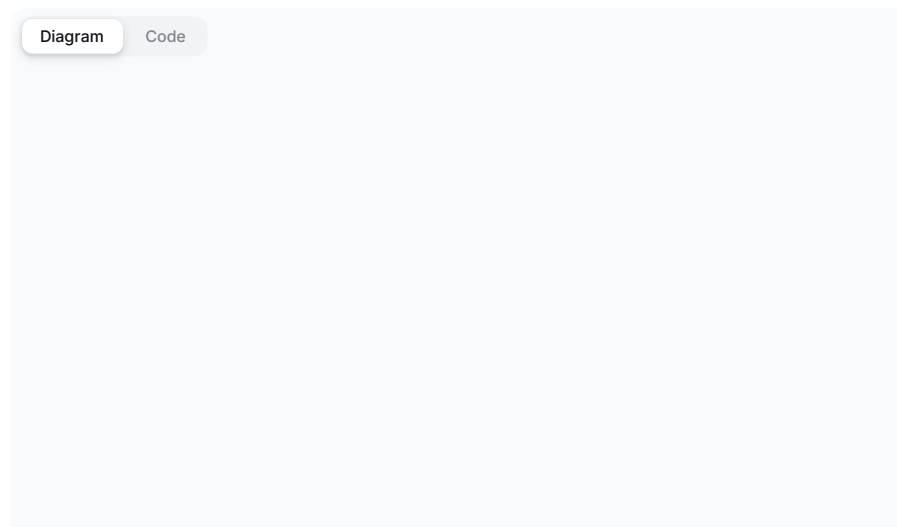
- **Main Thread = Waiter** (customer se baat karta hai, order leta hai, khana serve karta hai — fast & responsive)
- **Web Worker = Chef** (kitchen mein chup-chaap 10 minute khana banata hai, beech mein customer ko dikhta nahi)

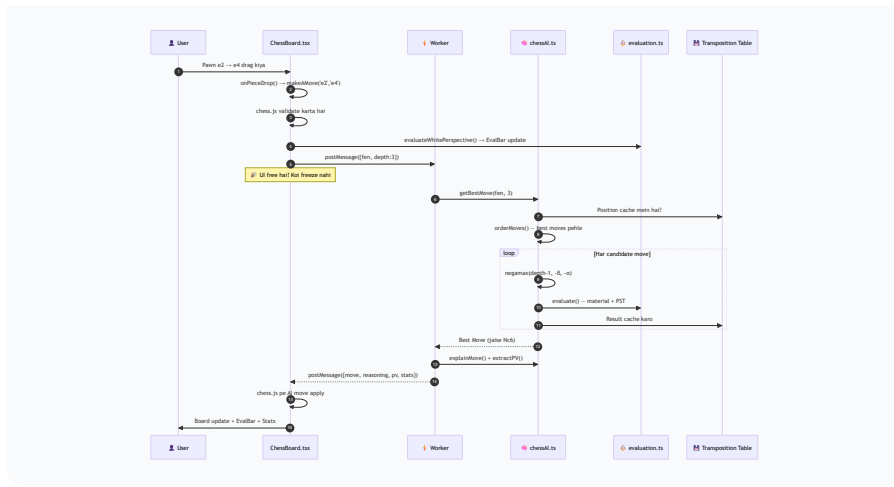
Agar waiter khud kitchen mein jaake khana banane lage, toh baaki customers kaun sambhalega? 🤪



📺 PART 1 — Ek Move Ka Pura Safar (Step-by-Step)

Maan lo tumne **1. e4** chala. Ab kya hoga? Chalo **12 steps** mein dekhte hain.





Steps Detail Mein:

#	Kya hota hai	Function
1	Piece drag-drop detect	<code>onPieceDrop()</code>
2	Move validate + execute	<code>makeAMove()</code>
3	Score calculate (White ke nazariye se)	<code>evaluateWhitePerspective()</code>
4	Worker ko FEN bhejo	<code>postMessage()</code>
5	Worker message catch kare	<code>self.onmessage</code>
6	Search start	<code>getBestMove()</code>
7	Cache check	<code>transpositionTable.get()</code>
8	Moves sort karo	<code>orderMoves()</code>
9	Recursive search	<code>negamax()</code>
10	Leaf pe capture search	<code>quiesce()</code>
11	Reasoning + PV extract	<code>explainMove()</code> , <code>extractPV()</code>
12	UI ko wapas bhejo	<code>postMessage()</code>

🧠 PART 2 — Algorithms Ek-Ek Karke (Deep Dive)

Ab main har algorithm ko analogy + diagram + code ke saath samjhata hoon.

🔹 Algorithm 1: Negamax (The Heart of Everything)

🌀 Kya Karta Hai?

Ye recursive function hai jo batata hai: "Is position mein, jiski turn hai, uske liye best score kya hai?"

🤔 Ye Negamax Kyu Kehlata Hai? (Minimax se alag kaise?)

Minimax mein 2 functions hote hain: `maximize()` (mera turn) aur `minimize()` (opponent ka turn).

Negamax kehta hai: *"Bhai 2 function kyu? Ek hi kaam chalega!"*

Trick: Zero-sum game mein — **mera profit = opponent ka loss.**

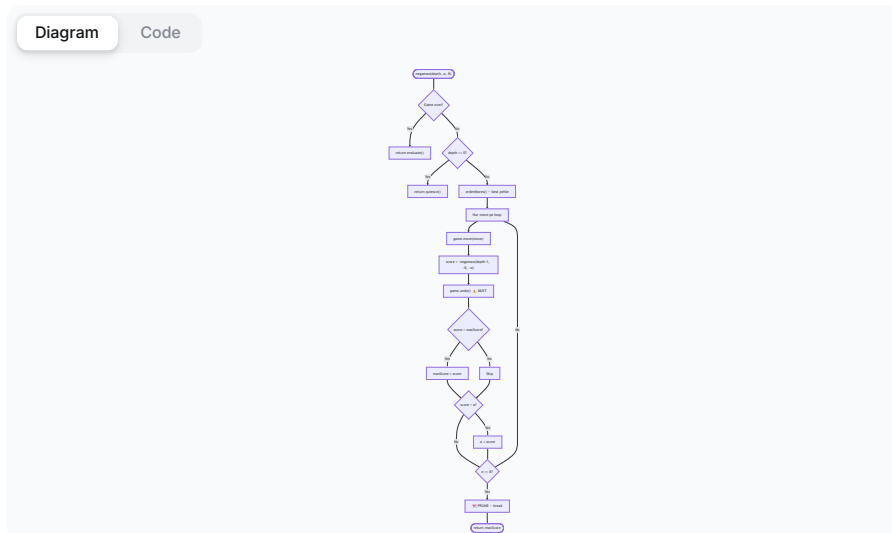
$$\max(A, B) = -\min(-A, -B)$$

Matlab:

- White ka score = `+300`
- Black ka score = `-300`
- Ek hi function dono colors ke liye kaam karega, bas **sign flip** karke.

Analogy: Do dost hain, ek hi pizza. Tum 3 slice loge toh dost ko 5 milega, ya tum 5 loge toh dost ko 3. **Tumhara gain = dost ka loss.** Isliye ek hi score likhna kaafi hai — jo tumhare perspective se hai.

 **Flowchart:**



 **Code:**

typescript

```
export function negamax(game, depth, alpha, beta, onNode) {
  if (game.isGameOver()) return evaluate(game);
  if (depth === 0) return quiesce(game, alpha, beta, onNode);

  let maxScore = -Infinity;
  const moves = orderMoves(game.moves({ verbose: true }));

  for (const move of moves) {
    game.move(move);
    // ⚠ Notice the MINUS sign - yahi Negamax ki jaan hai!
    const score = -negamax(game, depth - 1, -beta, -alpha, onNode);
    game.undo();

    if (score > maxScore) maxScore = score;
    if (score > alpha) alpha = score;
    if (alpha >= beta) break; // Alpha-Beta pruning
  }
  return maxScore;
}
```

📖 Mini Example (Depth 2, White's turn):

text

Position: White can play e4 or Nf3

Mera turn (White)

```
├─ e4 → Opponent ke paas: Nc6 ya e5
|     └─ Nc6 → score = -50 (White ke liye)
|       └─ e5 → score = +80 (White ke liye)
|         → Opponent best (-50 se best matlab White ke liye worst) = -50
|         → e4 ki value for White = -(-50) = +50 ✅
└─ Nf3 → Opponent ke paas: Nc6 ya d5
      └─ Nc6 → score = +30
        └─ d5 → score = +20
          → Opponent best = +20 (White ke liye best for opponent)
          → Nf3 ki value = -(+20) = -20 ❌
```

Best move = e4 (score +50) 🏆

🔹 Algorithm 2: Alpha-Beta Pruning (Search Fast Karne Ka Jadoo)

🤖 Kya Karta Hai?

Search tree ki **un branches ko skip** kar deta hai jahan already prove ho chuka hai ki "yahan se kuch acha nahi milega". ~70% time bach jaata hai, bina answer badle.

🧠 Two Magic Numbers:

- α (alpha): "Main (maximizer) already itna acha score jeet chuka hoon, isse kam nahi lunga"
- β (beta): "Opponent (minimizer) already itna kam score de raha hai, isse zyada nahi dega"

Rule: Agar $\alpha \geq \beta$, toh **cutoff** — matlab ye branch bekaar hai, aage dekhne ka faayda nahi.

Analogy: Tum dukaan pe ho, samaan dekh rahe ho.

- $\alpha = 500$ rupees (Itni best deal mil gayi, isse kam nahi lunga)
- $\beta = 700$ rupees (Shopkeeper itna hi max de sakta hai)
- Ab agar koi item **800 rupees** dikhaye toh? Matlab dekho — tumhara α (500) already shopkeeper ki limit (700) se compare mein... wait. Chalo simpler:

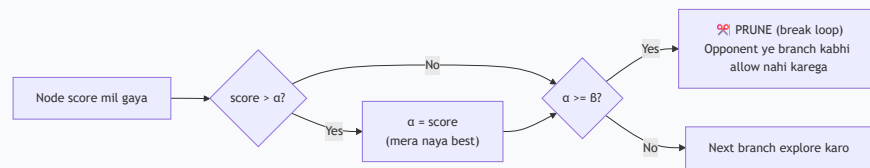
Asli analogy: Tum soch rahe ho "main A hostel jaunga ya B hostel".

- A hostel mein dekha: kamra acha hai, rent 5000 hai ($\alpha = 5000$ established)
- B hostel ka pehla kamra dekha: rent 8000, kuch khaas nahi
- B hostel ka doosra kamra dekhne jaane se pehle: **already pata hai pehla 8000 ka tha**, agar doosra 5000 se acha hoga toh bhi B ka average 5000 se upar hi rahega
- **Toh B ke baaki kamre dekhne ka kya faayda? → PRUNE B! ✅**

📊 Flowchart:

Diagram

Code



Code:

typescript

```

if (score > alpha) {
  alpha = score; // Naya best mila
}
if (alpha >= beta) {
  break; // ❌ CUTOFF! Baaki moves check karne ka koi matlab nahi
}
  
```

◆ Algorithm 3: Quiescence Search (Horizon Effect Killer)

🎯 Problem Kya Hai?

Maan lo depth 3 pe pahunch gaye. Last move mein **opponent tumhara Queen capture** kar leta hai — but depth 0 ho gayi. Engine sochta hai *"arre, main toh aage hi nahi dekh raha, ab score count karte hain"*.

Ye ghazab ki galti hai! Jaise fight ke beech mein score count karna — **aakhri punch ke baad hi count karo!**

🔧 Solution:

Depth 0 pe pahunchne ke baad bhi, **agar captures available hain**, toh unhe search karo. Jab tak board "quiet" (koi capture nahi) na ho jaye.

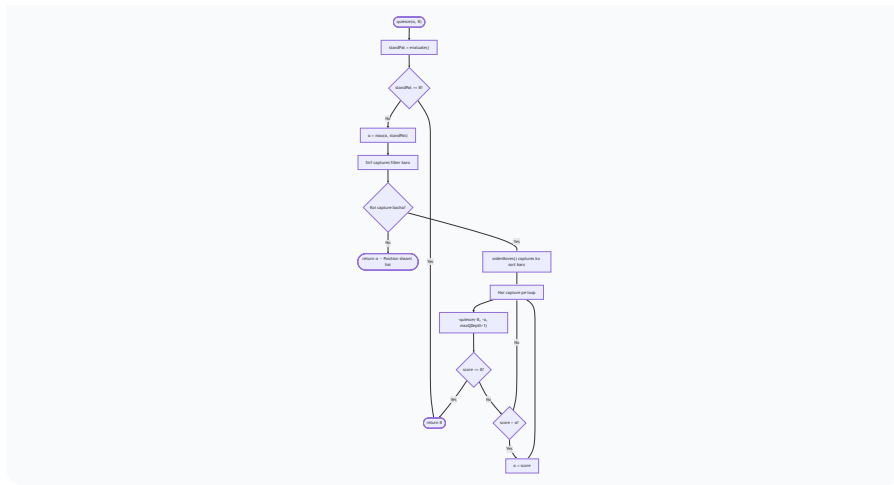
Analogy: Cricket match mein tum 1st innings ke score pe khush ho rahe the...

- **Bina Quiescence:** "50 runs ho gaye, main jeet gaya" 🙌
- **With Quiescence:** "Bhai, 2nd innings abhi baaki hai, chalo poora match khatam karte hain phir bolenge" 🏏

📊 Flowchart:

Diagram

Code



Code:

typescript

```

export function quiesce(game, alpha, beta, onNode, maxQDepth = 6) {
  if (game.isGameOver()) return evaluate(game);

  // Stand-pat: "Agar main kuch nahi karun toh kya score hai?"
  const standPat = evaluate(game);
  if (standPat >= beta) return beta; // Opponent allow nahi karega
  if (standPat > alpha) alpha = standPat;
  if (maxQDepth <= 0) return alpha; // Safety Limit

  // Sirf captures search karo (quiet moves skip)
  const candidateMoves = game.inCheck()
    ? game.moves({ verbose: true })
    : game.moves({ verbose: true }).filter((m) => !!m.captured);

  for (const move of orderMoves(candidateMoves)) {
    game.move(move);
    const score = -quiesce(game, -beta, -alpha, onNode, maxQDepth - 1);
    game.undo();

    if (score >= beta) return beta;
    if (score > alpha) alpha = score;
  }
  return alpha;
}

```

Note: Check mein ho toh **saare moves** dekhe jaate hain (kyunki check se nikalna zaroori hai, capture ho ya na ho).

◆ Algorithm 4: Move Ordering (MVV-LVA)

🔗 Kya Karta Hai?

Moves ko **smartly sort** karta hai — best moves sabse pehle.

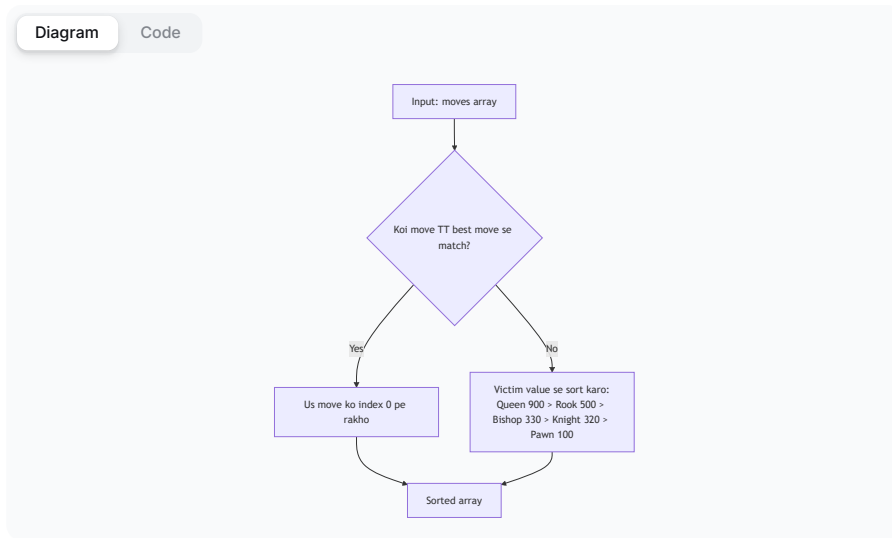
Kyu zaroori hai? Kyunki Alpha-Beta tab sabse fast chalta hai jab **best move** pehle try karo. Cutoff pehle hi ho jaata hai, baaki branches skip ho jaati hain.

MVV-LVA = Most Valuable Victim — Least Valuable Attacker

Analogy: Tumhare paas 5 punch hain, opponent ke saamne khada hai.

- **Smart order:** Pehle **jaw pe full power punch** maaro (ko ho sakta hai opponent gir jaye)
- **Dumb order:** Pehle **halka sa jhatka** maaro, phir dekho kya hota hai

Flowchart:



Code:

```

typescript

export function orderMoves(moves, ttMove) {
  return [...moves].sort((a, b) => {
    // 1. TT move = highest priority
    if (ttMove) {
      const aIsTT = a.lan === ttMove.lan;
      const bIsTT = b.lan === ttMove.lan;
      if (aIsTT && !bIsTT) return -1;
      if (!aIsTT && bIsTT) return 1;
    }
    // 2. MVV: bade victim wale captures pehle
    const aVal = a.captured ? PIECE_VALUES[a.captured] : 0;
    const bVal = b.captured ? PIECE_VALUES[b.captured] : 0;
    return bVal - aVal;
  });
}

```

Example Sort:

```

text

Input: [e4, Nxe5 (knight captures pawn), Qxf7 (queen captures pawn), Nf3]
Output: [Qxf7, Nxe5, e4, Nf3]
      ↑ Queen capture (pawn = 100, but big attacker) – wait

```

Hmm, dekho — MVV-LVA actually **victim ki value** dekhta hai, attacker ki nahi. Toh:

```

text

Qxf7 → captured = pawn (100) → score 100
Nxe5 → captured = pawn (100) → score 100

```

Dono same score hain, toh original order maintain hoga.

Real MVV-LVA mein attacker ki value bhi count hoti hai (least valuable attacker best hota hai — kyunki agar wapas capture ho gaya toh kam loss). Ye code simplified version hai, sirf victim value dekhta hai.

◆ **Algorithm 5: Transposition Table (Caching = Speed)**

🔧 **Problem:**

Chess mein **transposition** bahut common hai. Same position tak **alag-alag move orders** se pahunch sakte ho:

```
text

Path 1: 1. d4 Nf6 2. c4
Path 2: 1. c4 Nf6 2. d4
      ↑ Same position!
```

Bina cache ke, engine **same position ko 2 baar search** karega — waste!

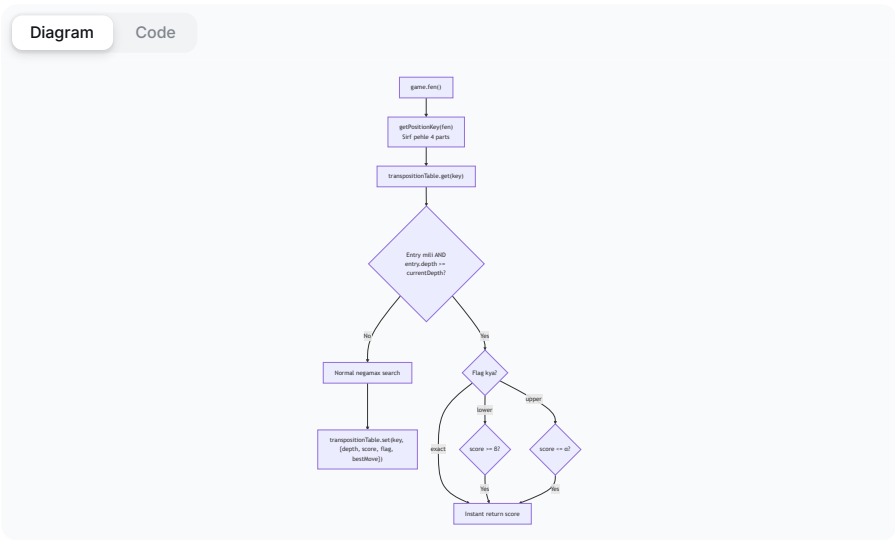
🔧 **Solution:**

Har position ki **FEN hash** → **score** memory mein store karo. Agar same position dobara aaye, **instant return**.

Analogy: Tumhare paas maths ke difficult sawaal ka solution **rough copy** mein likha hai. Exam mein same sawaal aaya toh **pura solve karne ki zaroorat nahi** — copy se answer likh do. Time bachao! 🕒

💡 **TT Entry Ke 3 Flags (Ye Important Hai):**

Flag	Matlab	Kab Use Hota Hai
exact	Pura accurate score	Normal case
lower	Score kam se kam itna toh hai (β cutoff hua tha)	Search mein prune hua
upper	Score zyada se zyada itna hai (α fail-low)	Search incomplete



📄 **Code:**

typescript

```
// CHECK (negamax ke starting mein):
const key = getPositionKey(game.fen());
const entry = transpositionTable.get(key);

if (entry && entry.depth >= depth) {
  if (entry.flag === 'exact') return entry.score;
  if (entry.flag === 'lower' && entry.score >= beta) return entry.score;
  if (entry.flag === 'upper' && entry.score <= alpha) return entry.score;
}

// STORE (Loop ke baad):
let flag: TTFlag = 'exact';
if (maxScore <= origAlpha) flag = 'upper';
else if (maxScore >= beta) flag = 'lower';

transpositionTable.set(key, { depth, score: maxScore, flag, bestMove });
```

🔑 `getPositionKey()` Ka Trick:

typescript

```
export function getPositionKey(fen: string): string {
  return fen.split(' ').slice(0, 4).join(' ');
  // ↑ Clocks (parts 4,5) hata diye!
}
```

Kyu? Kyunki same position **alag move numbers** pe bhi identical hoti hai. Agar clocks include karega toh cache **kabhi hit nahi hoga**.

◆ Algorithm 6: Evaluation (Engine Ki Aankhein)

🔧 Kya Karta Hai?

Position ka **score** deta hai (centipawns mein — 100 = 1 pawn).

2 Components:

1. **Material:** Pieces ki values
2. **PST (Piece-Square Tables):** Piece kahan khada hai, uska bonus/penalty

🎨 Piece Values (Standard):

Piece	Value (centipawns)
Pawn (p)	100
Knight (n)	320
Bishop (b)	330
Rook (r)	500
Queen (q)	900
King (k)	∞ (game over)

🔧 PST Kya Hai?

Har piece ke liye **8x8 grid** hoti hai jisme har square ka bonus likha hai.

Knight PST (example):

text

```
-50 -40 -30 -30 -30 -30 -40 -50
-40 -20  0  0  0  0 -20 -40
-30  0 10 15 15 10  0 -30
-30  5 15 20 20 15  5 -30
-30  0 15 20 20 15  0 -30
-30  5 10 15 15 10  5 -30
-40 -20  0  5  5  0 -20 -40
-50 -40 -30 -30 -30 -30 -40 -50
```

Dekho — **center mein +20 bonus, corner mein -50 penalty.**

Analogy: Soldier gali ke corner mein khada hai toh bekaar hai — kuch nahi kar sakta.

Chowk (center) pe khada hai toh pura area control karega! 🇮🇳

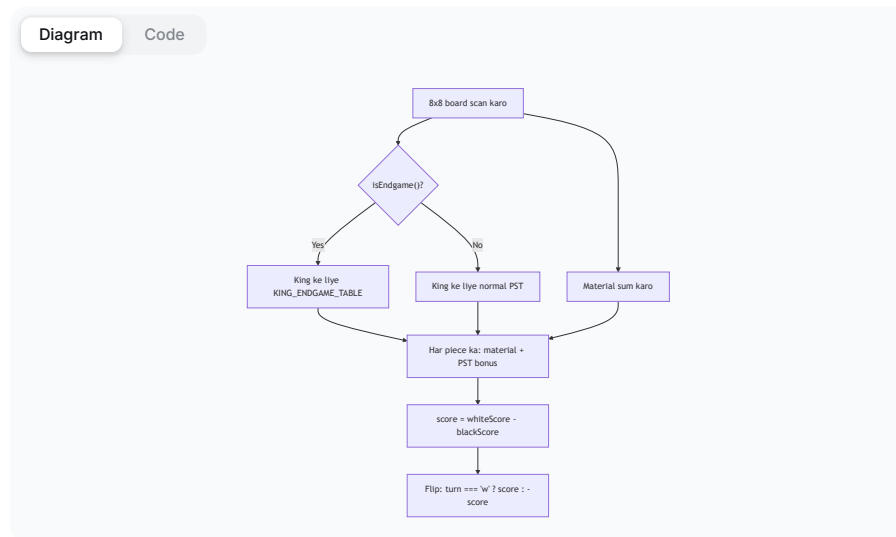
🇮🇳 **Endgame King Table Alag Hoti Hai:**

Opening/Middle game: King **corner mein safe** rehta hai (PST mein corner bonus)

Endgame: King **center mein active** hona chahiye (attack + support ke liye)

Isliye `isEndgame()` check hota hai — queens trade ho gayi? Toh `KING_ENDGAME_TABLE` use karo.

🇮🇳 **Flowchart:**



🇮🇳 **Code:**

typescript

```
export function evaluate(game: Chess): number {
  if (game.isCheckmate()) return -Infinity;
  if (game.isDraw()) return 0;

  let whiteScore = 0, blackScore = 0;
  const board = game.board();
  const endgame = isEndgame(game);

  for (let r = 0; r < 8; r++) {
    for (let c = 0; c < 8; c++) {
      const piece = board[r][c];
      if (!piece) continue;
```

```

const materialVal = PIECE_VALUES[piece.type] ?? 0;
const table = (piece.type === 'k' && endgame)
  ? KING_ENDGAME_TABLE
  : PST[piece.type];

// ⚠️ Black ke Liye row mirror karo (7 - r)
const pstVal = piece.color === 'w' ? table[r][c] : table[7 - r][c];
const totalVal = materialVal + pstVal;

if (piece.color === 'w') whiteScore += totalVal;
else blackScore += totalVal;
}
}

const score = whiteScore - blackScore;
// 🚨 CRITICAL: Side-to-move perspective!
return game.turn() === 'w' ? score : -score;
}

```

⚠️ Warning (Ye Galti Sab Karte Hain):

Agar tum **last line** bhool gaye, toh engine **suicide moves** chalega! Kyunki Negamax expect karta hai ki *"score hamesha current player ke liye positive ho"*.

◆ Algorithm 7: Iterative Deepening (Time Management)

🔧 Problem:

Direct Depth 5 search karoge toh kabhi 2 sec, kabhi 30 sec lag sakta hai. **Time predict nahi hota.**

🔧 Solution:

Pehle **Depth 1**, phir **Depth 2**, phir **Depth 3**... Har step pe timer check karo. Jab time khatam, **pichli complete depth ka best move** return kar do.

Bonus: Pichle depth ka best move agli depth pe **index 0** pe rakho — isse search super fast hoti hai!

Analogy: Exam mein tum:

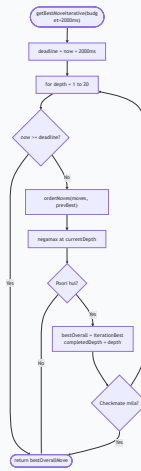
1. Pehle saare **1-mark wale** pakke karo ✅
2. Phir **2-mark wale** ✅
3. Phir **5-mark wale** (agar time bache) ⚠️

Agar time khatam, tumhare paas **kam se kam 1-mark wale** **toh pakke hain** — zero nahi!

📊 Flowchart:

Diagram

Code



Code:

typescript

```

export function getBestMoveIterative(fen, timeLimitMs = 2000, maxDepth = 20) {
    const game = new Chess(fen);
    const deadline = Date.now() + timeLimitMs;

    let bestOverallMove = legalMoves[0];
    let isTimeUp = false;
    let totalNodes = 0;

    // Lightweight check: har 1024 nodes pe timer dekho
    const checkTime = (): boolean => {
        totalNodes++;
        if ((totalNodes & 1023) === 0 && Date.now() >= deadline) isTimeUp = true;
        return isTimeUp;
    };

    for (let depth = 1; depth <= maxDepth; depth++) {
        if (Date.now() >= deadline) break;

        // ⚠ Previous best move index 0 pe!
        const moves = orderMoves(legalMoves, bestOverallMove);
        let iterationCompleted = true;
        let iterationBestMove = moves[0];

        for (const move of moves) {
            if (checkTime()) { iterationCompleted = false; break; }

            game.move(move);
            const score = -negamax(game, depth - 1, -Infinity, Infinity, checkTime);
            game.undo();

            if (isTimeUp) { iterationCompleted = false; break; }
            if (score > bestScore) { bestScore = score; iterationBestMove = move; }
        }

        if (iterationCompleted) {
            bestOverallMove = iterationBestMove;
            completedDepth = depth;
        } else {
            break; // Adhuri depth discard karo
        }
    }

    return { bestMove: bestOverallMove, depthReached: completedDepth };
}

```

(totalNodes & 1023) === 0 — Ye bitwise trick hai: har 1024 nodes pe true hota hai.
Date.now() slow hota hai, isliye bar-bar call nahi karte.

PART 3 — Core Data Structures

1 chessRef.current — Game Engine Ref

```
typescript  
  
const chessRef = useRef(new Chess());
```

Kya hai: chess.js ka live instance — rule validation, move generation, check detection.
Kyu useRef: React re-render nahi karna chahte. Performance.

Analogy: Ye tumhari **physical chess board** hai. Piece hilao — board update hoti hai.
Har baar naya board nahi banate.

2 fen State — UI Rendering

```
typescript  
  
const [fen, setFen] = useState("rnbqkbnr/...");
```

Kya hai: FEN string React state mein — taaki react-chessboard board redraw kar sake.

3 Transposition Table Map

```
typescript  
  
export interface TTEntry {  
  depth: number;  
  score: number;  
  flag: 'exact' | 'lower' | 'upper';  
  bestMove?: Move | null;  
}
```

Kya hai: Map<string, TTEntry> — FEN → cached score.

4 Principal Variation (PV)

```
typescript  
  
pv: string[] = ["Nc6", "Nc3", "Nf6"]
```

Kya hai: Anticipated moves ki line. UI mein "AI aage ye soch raha hai" dikhane ke liye.

5 Worker Messages

```
typescript  
  
// Main → Worker  
{ fen: string, depth?: number, timeLimitMs?: number }  
  
// Worker → Main  
{ move, bestMove, depthReached, score, nodes, timeMs, reasoning, pv }
```

⚠️ PART 4 — Beginner Gotchas (Ye Galtiyan Mat Karna!)

❌ Galti #1: `evaluate()` Ka Perspective Flip Bhoolna

```
typescript
// ❌ GALAT — Black ke Liye suicide moves
return whiteScore - blackScore;

// ✅ SAHI
const score = whiteScore - blackScore;
return game.turn() === 'w' ? score : -score;
```

Kyu? Negamax har node pe sochta hai *"main maximize kar raha hoon"*. Agar Black ke turn pe White's perspective return karoge, toh Black apne khilaaf khelega!

❌ Galti #2: `game.undo()` Miss Karna

```
typescript
// ❌ GALAT — Board corrupt
game.move(move);
const score = -negamax(depth - 1);
// undo miss! Board pe trial move permanently reh gaya

// ✅ SAHI
game.move(move);
const score = -negamax(depth - 1, -beta, -alpha);
game.undo(); // ← Ye must hai
```

Kyu? Search mein hazaaron trial moves chalte hain. Har branch ke baad board restore karna zaroori hai, warna agli branch galat position pe search karegi.

Analogy: Tum kisi room mein furniture hilake dekh rahe ho — *"sofa yahan acha lagega kya?"*. Dekhne ke baad **wapas original jagah** rakhna zaroori hai. Warna ghar ka sab kuch bikhra rahega. 🏠

❌ Galti #3: Loop Mein `new Chess(fen)` Banana

```
typescript
// ❌ GALAT — 10x-20x slow!
for (const m of moves) {
  const nextGame = new Chess(game.fen());
  nextGame.move(m);
}

// ✅ SAHI — Ek hi instance pe mutate + undo
for (const m of moves) {
  game.move(m);
  // ... search ...
  game.undo();
}
```

Kyu? Har iteration mein `new Chess()` = 1000s of objects allocate. Search millions of nodes karta hai → **crash level slow**.

❌ Galti #4: FEN Clocks TT Key Mein Daalna

```
typescript

// ❌ GALAT - Cache kabhi hit nahi hoga
const key = fen;

// ✅ SAHI - Sirf pehle 4 parts
const key = fen.split(' ').slice(0, 4).join(' ');
```

❌ Galti #5: Main Thread Pe Direct Search Chalana

```
typescript

// ❌ GALAT - UI freeze!
const bestMove = getBestMove(fen, 4); // 2000ms freeze 🤯

// ✅ SAHI - Worker mein bhejo
workerRef.current?.postMessage({ fen, depth: 4 });
```

🚀 PART 5 — Naya Feature Kahan Add Karein?

Feature Idea	File	Function
Bishop pair bonus, Doubled pawns penalty	evaluation.ts	evaluate()
Killer moves, History heuristic	chessAI.ts	orderMoves(), negamax()
Naya telemetry stat (TT hits count)	worker + UI	chessAI.worker.ts + AIStats.tsx
Naya difficulty level	ChessBoard.tsx	difficulty selector
Better explainMove (fork, pin detection)	chessAI.ts	explainMove()
Sound effects	ChessBoard.tsx	makeAMove()

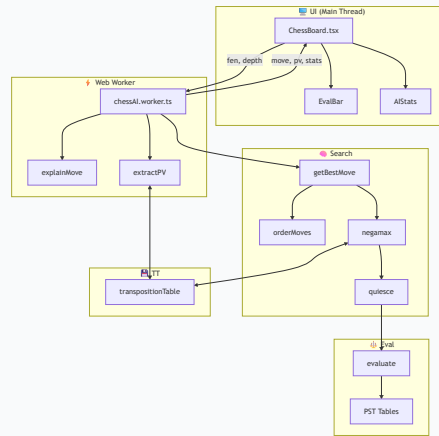
🎯 TL;DR — Ek Line Mein

User ke move pe, Web Worker background mein FEN lekar Negamax search chalata hai. Har node pe Piece-Square Tables se score milta hai, Transposition Table se purani positions reuse hoti hain, Alpha-Beta se useless branches prune hoti hain, aur Quiescence se capture sequences shaant hoti hain — result UI ko wapas jaata hai bina screen freeze kiye.

🏁 Poora System Ek Nazar Mein

Diagram

Code



Bhai, agar ab bhi kuch clear nahi hua — batao **kis part pe atke ho**, main usko **aur simple analogy** ke saath samjha dunga! 🚀

This response is AI-generated, for reference only.